SQL PRACTICE | TASK 4

Data Modification & Integrity

Complete solutions - MySQL Employees sample database

Practice safely. Use sandbox tables only. The expected outputs below assume the stated records and constraints exist in your database.

1 Create a practice table

```sql
CREATE TABLE employee_sandbox AS
SELECT * FROM employees
LIMIT 1000;
```

2 INSERT - Add new employees

```sql
-- Insert one employee
INSERT INTO employee_sandbox
(emp_no, birth_date, first_name, last_name, gender, hire_date)
VALUES
(500001, '1995-08-14', 'Emma', 'Johnson', 'F', '2023-06-01');

-- Insert multiple employees
INSERT INTO employee_sandbox VALUES
(500002, '1990-11-03', 'Liam', 'Smith', 'M', '2023-06-01'),
(500003, '1988-04-22', 'Olivia', 'Brown', 'F', '2023-06-01');

-- Verify
SELECT * FROM employee_sandbox
WHERE emp_no >= 500000;
```

Explanation INSERT adds rows; SELECT verifies the inserted test records.

3 UPDATE - Modify existing records

```sql
UPDATE employee_sandbox
SET first_name = 'Amelia'
WHERE emp_no = 500003;

UPDATE employee_sandbox
SET hire_date = '2023-07-15'
WHERE first_name = 'Georgi'
  AND last_name = 'Facello';

-- Verify
SELECT * FROM employee_sandbox
WHERE first_name IN ('Amelia', 'Georgi');
```

Explanation UPDATE changes only rows matching the WHERE condition.

4 DELETE - Remove test records

```sql
DELETE FROM employee_sandbox
WHERE hire_date = '2023-06-01'
  AND emp_no > 500000;

SELECT COUNT(*) FROM employee_sandbox
WHERE emp_no > 500000;
```

Expected count: 0 if all records with employee numbers above 500000 were the test records inserted earlier.

5 Primary key constraint

```sql
INSERT INTO employee_sandbox
VALUES (10001, '1953-09-02', 'John', 'Doe', 'M', '1986-06-26');
```

Expected duplicate-entry error if employee number 10001 already exists as the primary key. A primary key must be unique.

6 Foreign key constraint

```sql
CREATE TABLE salary_sandbox AS
SELECT * FROM salaries
LIMIT 1000;

ALTER TABLE salary_sandbox
ADD PRIMARY KEY (emp_no, from_date);

ALTER TABLE salary_sandbox
ADD CONSTRAINT fk_emp
FOREIGN KEY (emp_no)
REFERENCES employee_sandbox(emp_no);

-- Test an employee number that does not exist
INSERT INTO salary_sandbox
VALUES (999999, 80000, '2023-01-01', '9999-01-01');
```

Expected foreign-key error if employee 999999 does not exist in employee_sandbox. The salary row must reference an existing employee.

7 Transaction control - ROLLBACK

```sql
START TRANSACTION;

UPDATE employee_sandbox
SET first_name = 'Alexander'
WHERE emp_no = 10002;

INSERT INTO salary_sandbox
VALUES (10002, 75000, CURDATE(), '9999-01-01');

SELECT * FROM employee_sandbox WHERE emp_no = 10002;
SELECT * FROM salary_sandbox WHERE emp_no = 10002;

ROLLBACK;

SELECT * FROM employee_sandbox WHERE emp_no = 10002;
```

Note ROLLBACK undoes uncommitted changes when the tables use a transactional storage engine and no implicit commit occurs.

8 Savepoints

```sql
START TRANSACTION;

UPDATE employee_sandbox
SET hire_date = '2023-01-01'
WHERE emp_no = 10003;

SAVEPOINT sp1;

DELETE FROM employee_sandbox
WHERE first_name LIKE 'A%';

ROLLBACK TO SAVEPOINT sp1;
COMMIT;
```

Result The deletion is undone; the hire-date update remains and is committed.

9 Data validation - Python pseudocode

```python
def update_employee(emp_no, new_name):
    old_record = execute_query(
        "SELECT * FROM employee_sandbox WHERE emp_no = ?",
        emp_no
    )
    execute_query(
        "UPDATE employee_sandbox SET first_name = ? WHERE emp_no = ?",
        (new_name, emp_no)
    )
    new_record = execute_query(
        "SELECT * FROM employee_sandbox WHERE emp_no = ?",
        emp_no
    )
    if old_record['gender'] != new_record['gender']:
        rollback_changes()
        raise IntegrityError("Gender cannot be modified")
    log_change(old_record, new_record)
```

Note This is pseudocode: execute_query(), rollback_changes(), and IntegrityError must be supplied by the actual application/database code.

10 Data consistency check

```sql
CREATE TABLE pre_update AS
SELECT * FROM employee_sandbox;

SELECT
  (SELECT COUNT(*) FROM pre_update) AS old_count,
  (SELECT COUNT(*) FROM employee_sandbox) AS new_count;
```

Explanation Compares the number of rows in the snapshot with the current table. Counts may differ after inserts or deletes.

11 Add a UNIQUE email constraint

```sql
ALTER TABLE employee_sandbox
ADD COLUMN email VARCHAR(255);

ALTER TABLE employee_sandbox
ADD CONSTRAINT uc_email UNIQUE (email);

UPDATE employee_sandbox
SET email = 'test@company.com'
WHERE emp_no = 10001;

UPDATE employee_sandbox
SET email = 'test@company.com'
WHERE emp_no = 10002;
```

Expected the second update fails if the first succeeds, because duplicate non-NULL email values are not allowed.

12 Remove and restore a foreign key

```sql
ALTER TABLE salary_sandbox
DROP FOREIGN KEY fk_emp;

LOAD DATA INFILE 'legacy_salaries.csv'
INTO TABLE salary_sandbox;

ALTER TABLE salary_sandbox
ADD CONSTRAINT fk_emp
FOREIGN KEY (emp_no)
REFERENCES employee_sandbox(emp_no)
ON DELETE CASCADE;
```

Note The CSV import requires a valid file path, suitable MySQL permissions, and compatible columns. Re-adding the foreign key fails if existing salary rows reference missing employees. ON DELETE CASCADE deletes related salary rows when an employee is deleted.

13 Real-world scenario - Department consolidation

```sql
START TRANSACTION;

UPDATE dept_emp
SET dept_no = (
    SELECT dept_no FROM departments
    WHERE dept_name = 'People Ops'
)
WHERE dept_no = (
    SELECT dept_no FROM departments
    WHERE dept_name = 'Human Resources'
)
AND to_date > CURDATE();

DELETE FROM departments
WHERE dept_name = 'Human Resources';

SELECT COUNT(*)
FROM dept_emp
WHERE dept_no = 'd007'
  AND to_date > CURDATE();

COMMIT;
```

Expected active HR count: 0 if all relevant records were transferred. The DELETE must satisfy foreign-key constraints, and the supplied schema must contain both department names.

14 Real-world scenario - Audit before data purge

```sql
CREATE TABLE deletion_audit (
    emp_no INT PRIMARY KEY,
    delete_date DATE
);

START TRANSACTION;

INSERT INTO deletion_audit
SELECT emp_no, CURDATE()
FROM employee_sandbox
WHERE hire_date < '1990-01-01';

DELETE FROM employee_sandbox
WHERE hire_date < '1990-01-01';

SELECT COUNT(*)
FROM employee_sandbox
WHERE hire_date < '1990-01-01';

COMMIT;
```

Expected count: 0 after a successful deletion. This example logs employee IDs and dates before deleting matching rows. Real data-purge workflows must also consider related tables and legal retention requirements.

Quick revision

Command	Purpose
`INSERT`	Adds records
`UPDATE`	Modifies records
`DELETE`	Removes records
`PRIMARY KEY`	Uniquely identifies each row
`FOREIGN KEY`	Maintains relationships between tables
`UNIQUE`	Prevents duplicate non-NULL values
`START TRANSACTION`	Begins a transaction
`COMMIT`	Saves transaction changes
`ROLLBACK`	Undoes uncommitted changes
`SAVEPOINT`	Creates a rollback point
`ALTER TABLE`	Changes table structure

Safety tip Run a SELECT with the same WHERE condition before executing UPDATE or DELETE. Use sandbox tables, not production data, for practice.